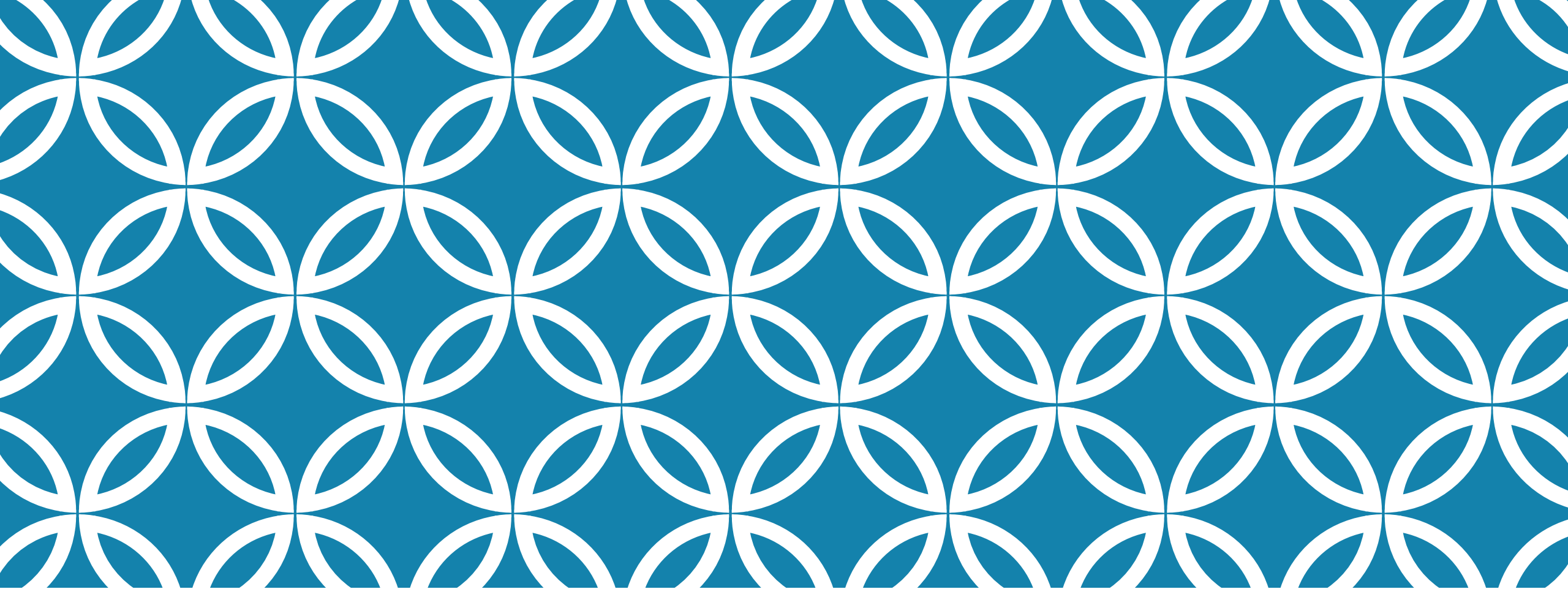


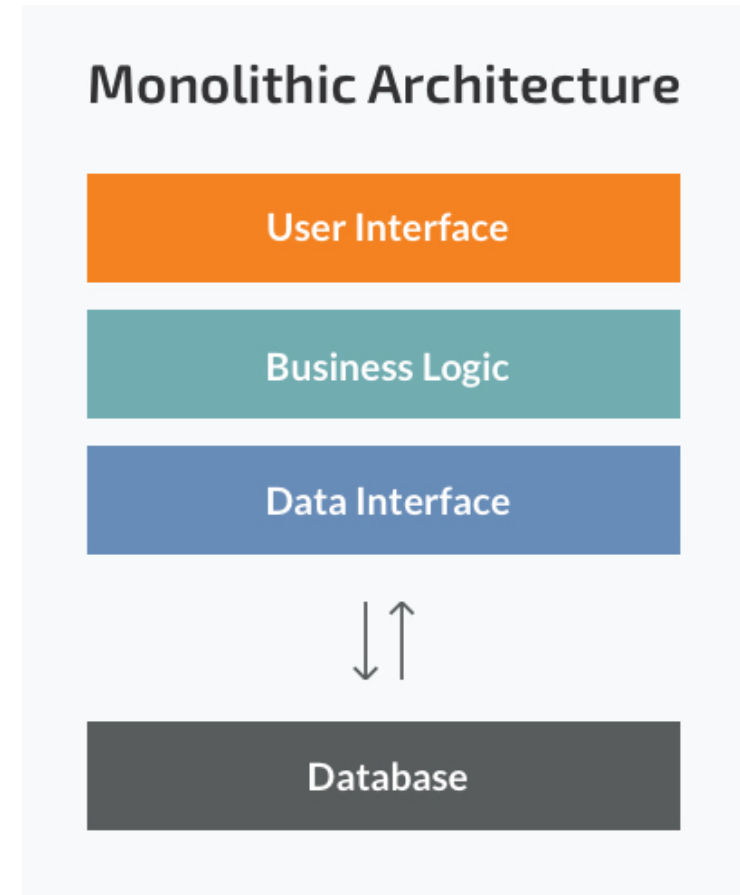


# MICROSERVICES ARCHITECTURE

Software Engineering Course – 06  
2025-2026

# WHAT WILL WE DO TODAY?

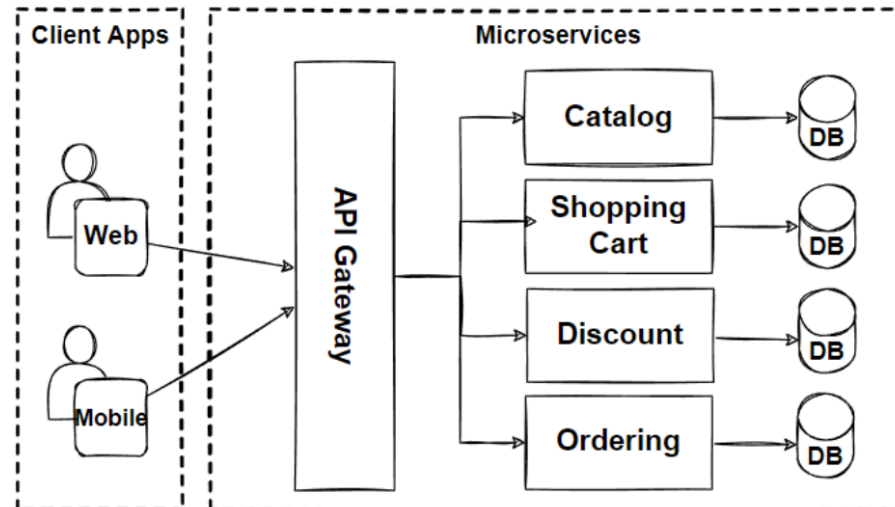
We're going to quickly review one software architecture style you're certainly familiar with – the **monolith** – then try to understand what its limitations are and how we can fix them.



[Image source](#)

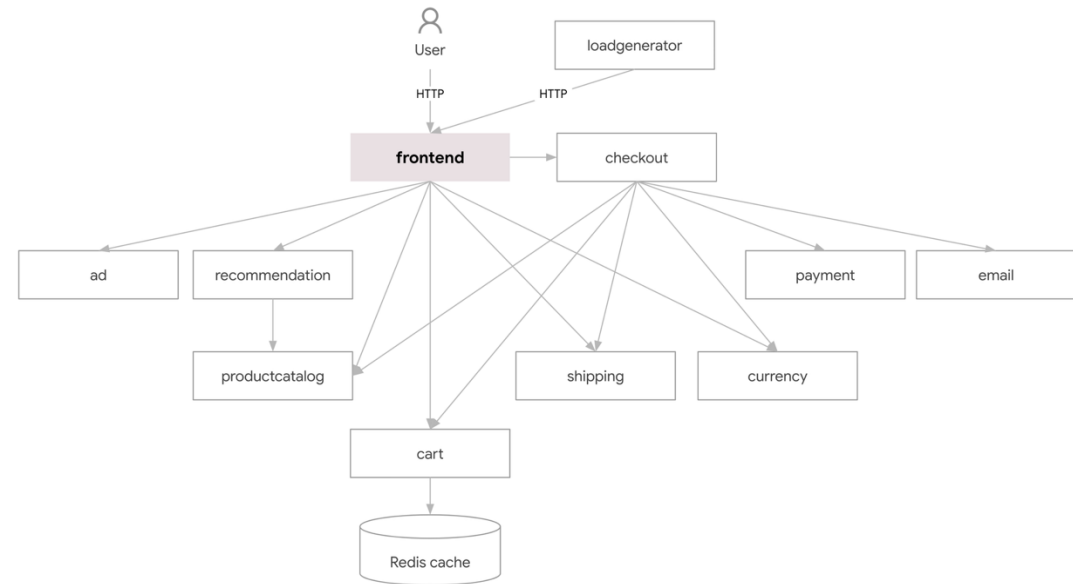
# WHAT WILL WE DO TODAY?

The most popular modern solution to the monolith's limitations is an architecture based on **microservices**. We will see what it involves, how we can use it to design and implement our own projects and what are the risks associated with it.

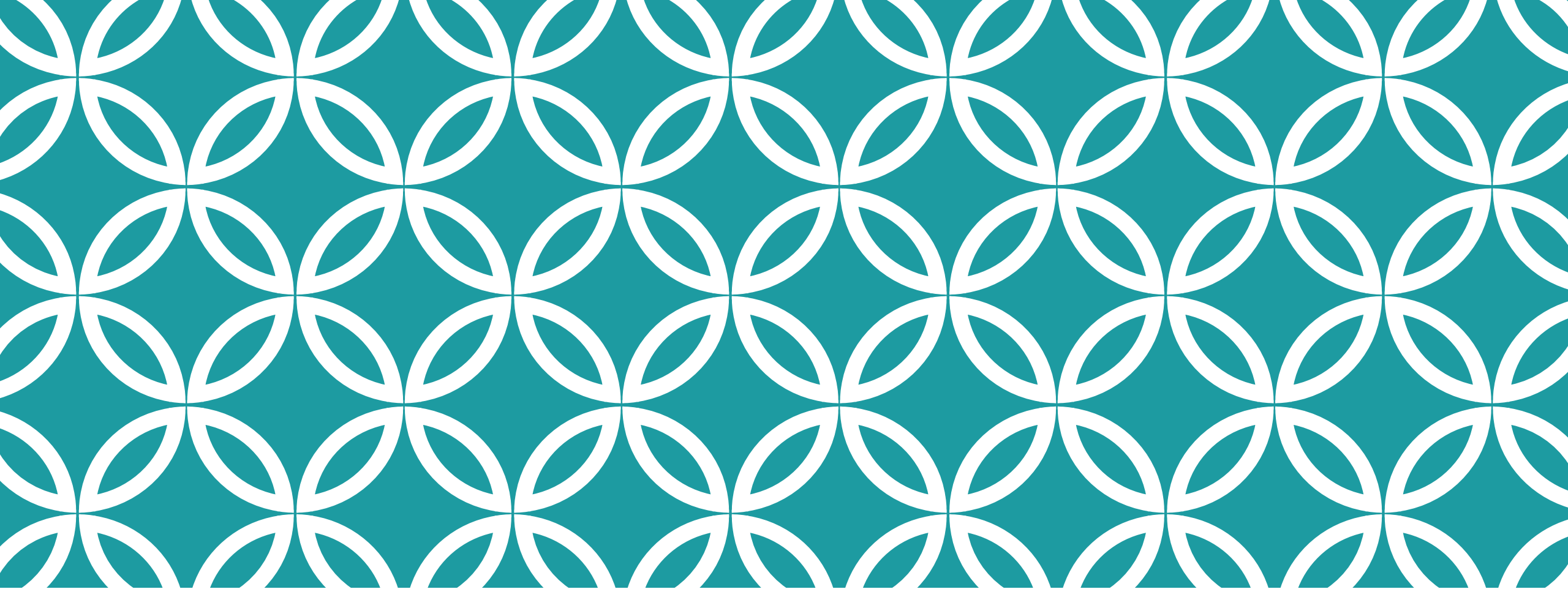


# WHAT WILL WE DO TODAY?

At the end, I'll show you how to **deploy** a microservices-based demo app using **Kubernetes** – and how you can do so yourself on your own laptop!



# kubernetes



# SHORT HISTORY OF APP ARCHITECTURES



# SHORT HISTORY OF APP ARCHITECTURES

The first computer programs ran on **mainframes** (think room-sized computers!), using punch cards to store instructions



# SHORT HISTORY OF APP ARCHITECTURES

Your program was probably **the only thing running on the system** – sometimes, there wasn't even an Operating System (OS) to speak of! The program had to handle things from **hardware setup** to **memory management** and more.

```
Menu  Utilities  Compilers  Options  Status  Help
-----
Option ==>
0  Settings      Terminal and user parameters      User ID . : SYSADM
1  View          Display source data or listings   Time. . . : 15:33
2  Edit          Create or change source data      Terminal. : 3278
3  Utilities     Perform utility functions         Screen. . : 1
4  Foreground    Interactive language processing   Language. : ENGLISH
5  Batch         Submit job for language processing Appl ID . : ISR
6  Command       Enter TSO or Workstation commands TSO logon : ISPFPROC
7  Dialog Test   Perform dialog testing            TSO prefix: SYSADM
9  IBM Products  IBM program development products  System ID : P390
10 SCLM          SW Configuration Library Manager  MVS acct. : ACCT#
11 Workplace    ISPF Object/Action Workplace     Release . : ISPF 5.2
M  More         Additional IBM Products

Enter X to Terminate using log/list defaults
```



# SHORT HISTORY OF APP ARCHITECTURES

In the 80's, the revolution of the **Personal Computers** (PCs) led to the development of operating systems with support for **Graphical User Interfaces** (GUIs)

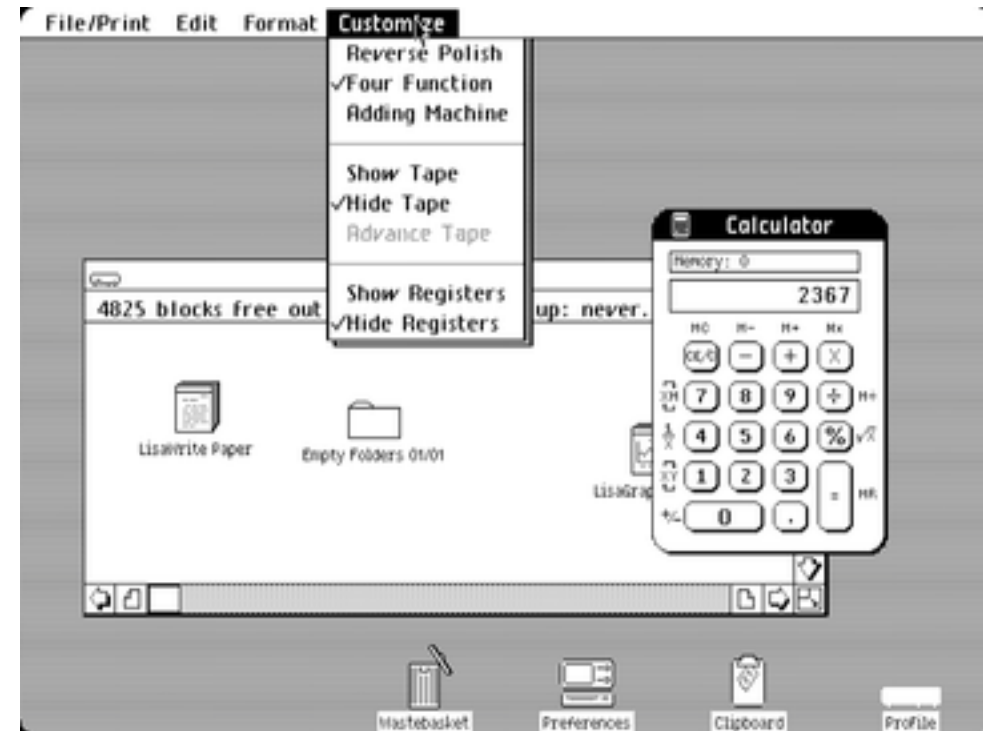


The original Macintosh



# SHORT HISTORY OF APP ARCHITECTURES

Applications were mostly **self-contained** and **highly specialized** (examples: calculator, text editor, word processor, filesystem navigator, assembler/compiler, text formatter, video game etc.)



Apple's Lisa Office System 3.1

# SHORT HISTORY OF APP ARCHITECTURES

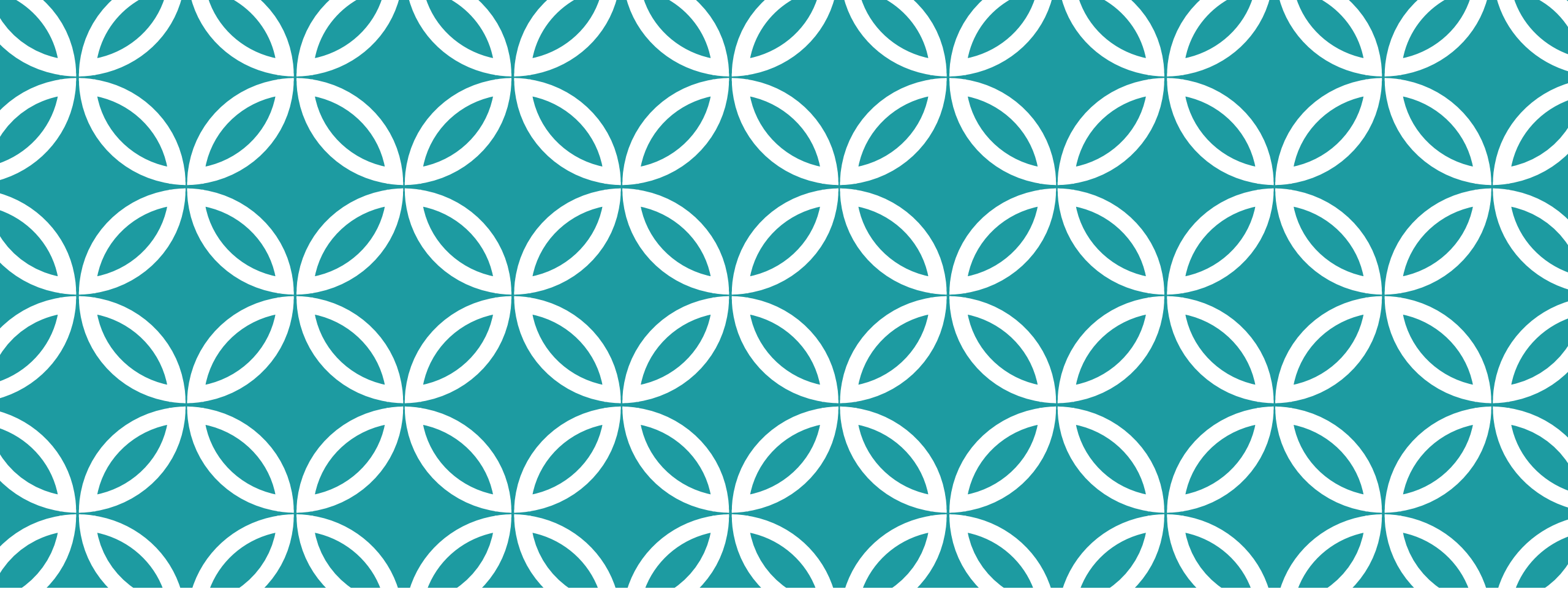
The 1990's saw the rise of a new computing paradigm – **networked computing** – and the largest public network of interconnected computers – the **Internet**.



# SHORT HISTORY OF APP ARCHITECTURES

As more people started using the internet, companies rushed to build an online presence – thus fostering the development of **web browsers** (**web clients**) and the offering of services across the network (**websites**, hosted on **web servers**).





# EARLY WEB APP ARCHITECTURES

Latest trends from 20 years ago

# EARLY WEB ARCHITECTURES

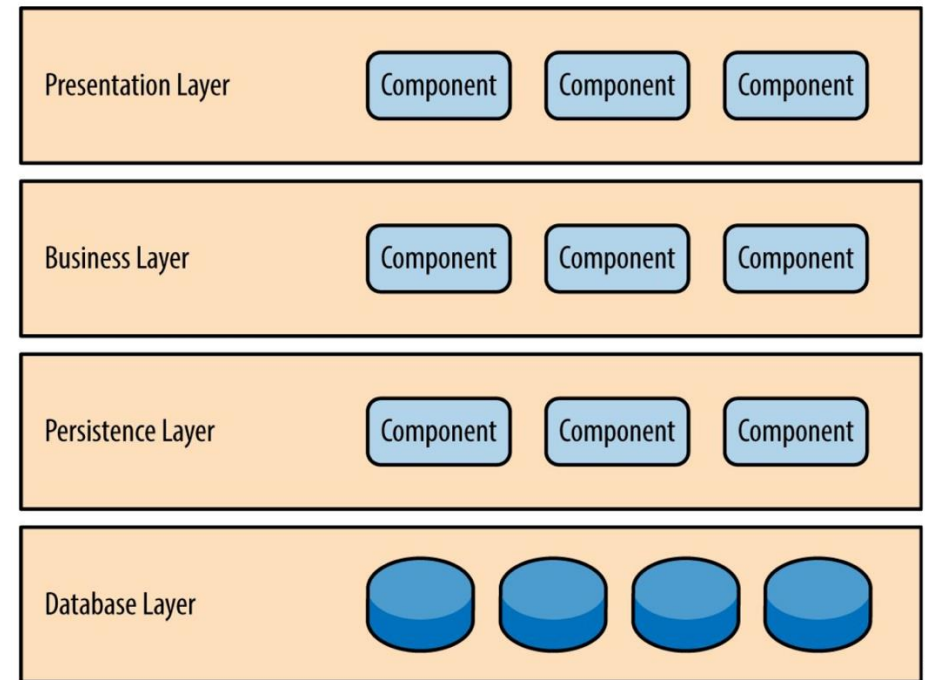
Unlike the offline-only programs of the '80s, which ran only locally and stored all their data on disk, web applications are based on web **clients** (browsers) connecting to remote **servers**, allowing users to interact with the content.

The **distributed** nature of the web and its **scale** forced programmers to adopt different architectural approaches to the design and implementation of software solutions.

# LAYERED ARCHITECTURE

One of the most common architectural styles is the **layered architecture**. The internet itself is a layered design (OSI Layers).

**Separation of concerns** is realized by ensuring every layer has a **specific functional responsibility** (e.g.: database layer communicates with the DB and provides an abstract data access interface to higher-level layers).



[Diagram source](#)

# LAYERED ARCHITECTURE

Can you give other examples of layered architectures?  
What happens when you need to add a new feature?

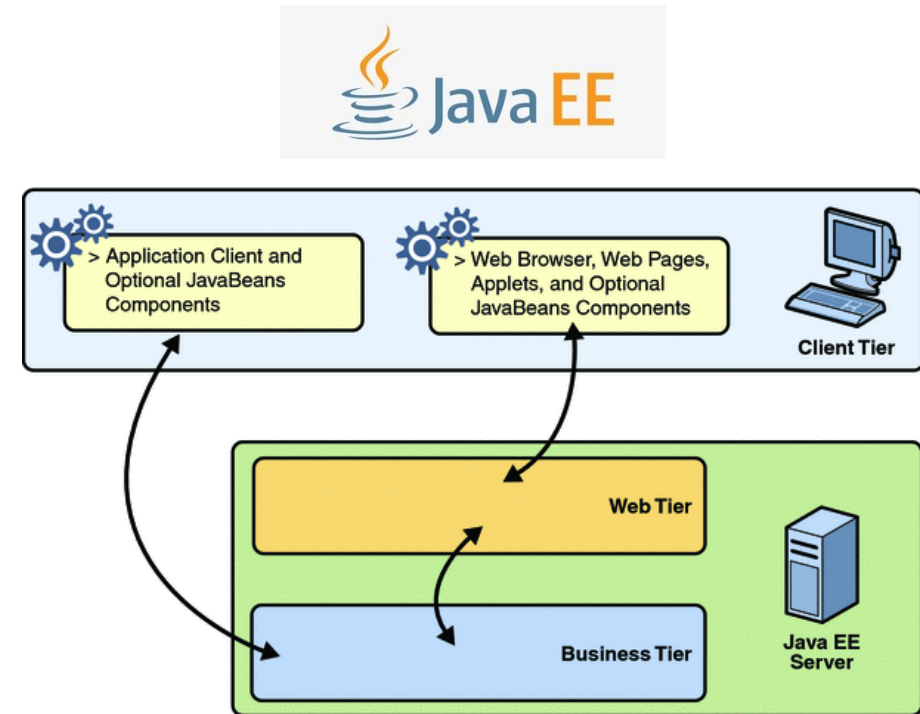


# DISTRIBUTED MULTITIERED APPLICATIONS

In the early 2000's, **Java desktop clients** (and **Java Applets**) were a popular way of developing **networked applications**.

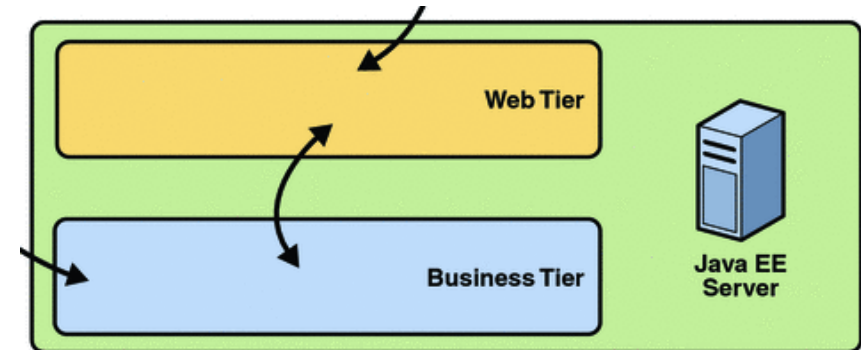
The clients were used simply as **interfaces** to connect to the **central server** where all the **business logic** and **data access** happened.

This is somewhat inspired by the access terminals of mainframes from the '60s.



# DISTRIBUTED MULTITIERED APPLICATIONS

**Note:** this "new" architecture contains elements of previous architectures (such as a layered architecture on the back end!)



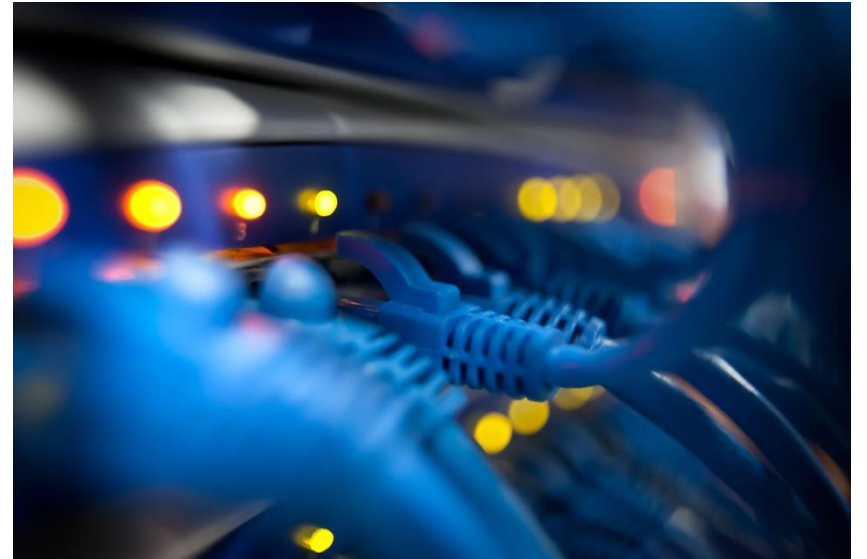
# DISTRIBUTED MULTITIERED APPLICATIONS

Can you think of some disadvantages with this approach?  
Which considerations/requirements led to this architecture?

# DISTRIBUTED MULTITIERED APPLICATIONS

One advantage of this architecture is that you ensure **loose coupling** of the user interface (client app) and the business logic (back end server), since there's a network gap between them.

Furthermore, the client app and central server can evolve and be updated **asynchronously**.



# LIMITATIONS OF DISTRIBUTED MULTITIERED APPLICATIONS

The last bit is also this architecture's **biggest issue**:

- you need to update the client software in anticipation for a change to the server's API
- the server must support previous versions of the API while older clients are still in use
- **there is global downtime when the central server needs to be stopped for upgrades**

# LIMITATIONS OF DISTRIBUTED MULTITIERED APPLICATIONS

Furthermore, while the architecture encourages loose coupling between the client and the server, there's no limit to how **tightly coupled** the modules on the front end or back end might be!

You could have a central server with 100.000 lines of code, serving 3 different client apps, each with 50.000 lines of code. It becomes hard to manage such a large project.

# DISTRIBUTED MULTITIERED APPLICATIONS

Think this architectural pattern is not used anymore?

You'd be surprised...

# MONOLITHIC ARCHITECTURE

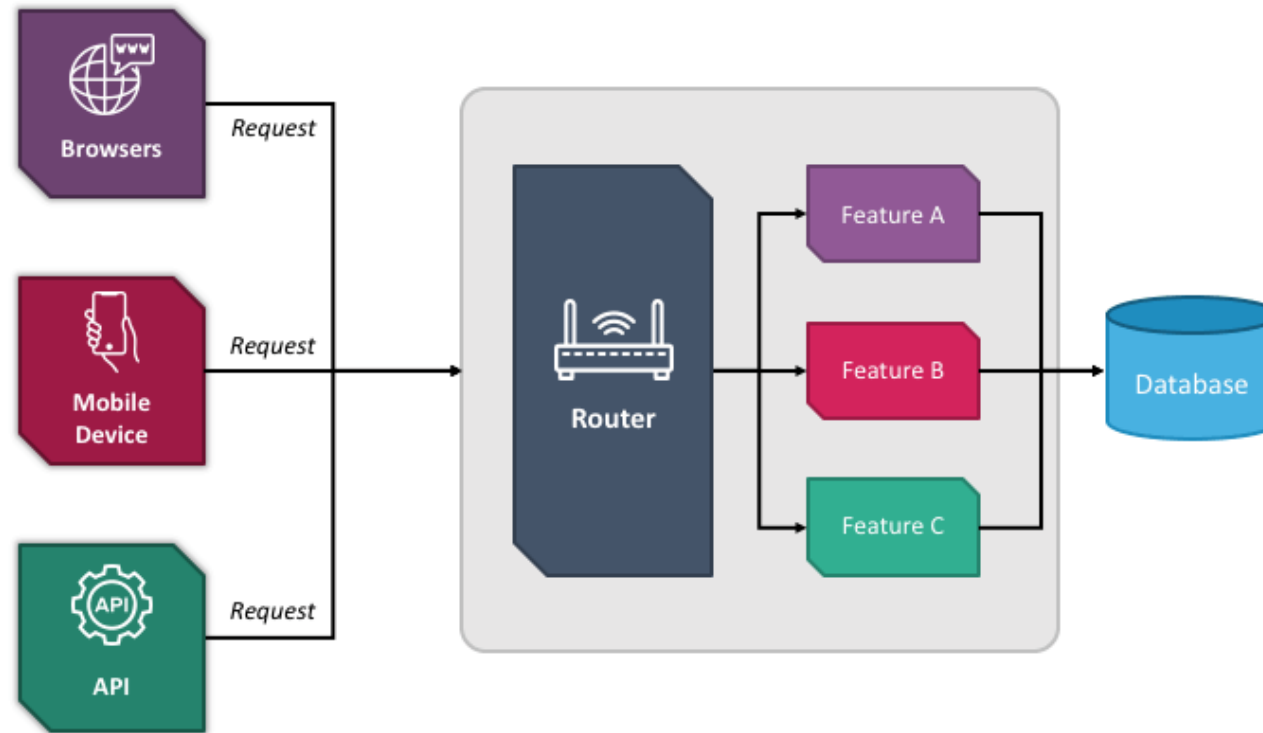
A lot of modern web applications follow a pattern like this architecture: there's a **front end client** (with the app's user interface) sending requests to a **back end server** (where all the business logic and data persistence is handled).

If you've ever built a simple web app using the Mode-View-Controller (MVC) pattern, you've (implicitly) used a **monolithic architecture**: there's only one software system, one process, handling all the business logic of the app.



# MONOLITHIC ARCHITECTURE

Monolithic Server



# MONOLITHIC ARCHITECTURE

Does this architecture describe the way you've built (web) apps until now?

Did you encounter any issues with it?

# MONOLITHIC ARCHITECTURE

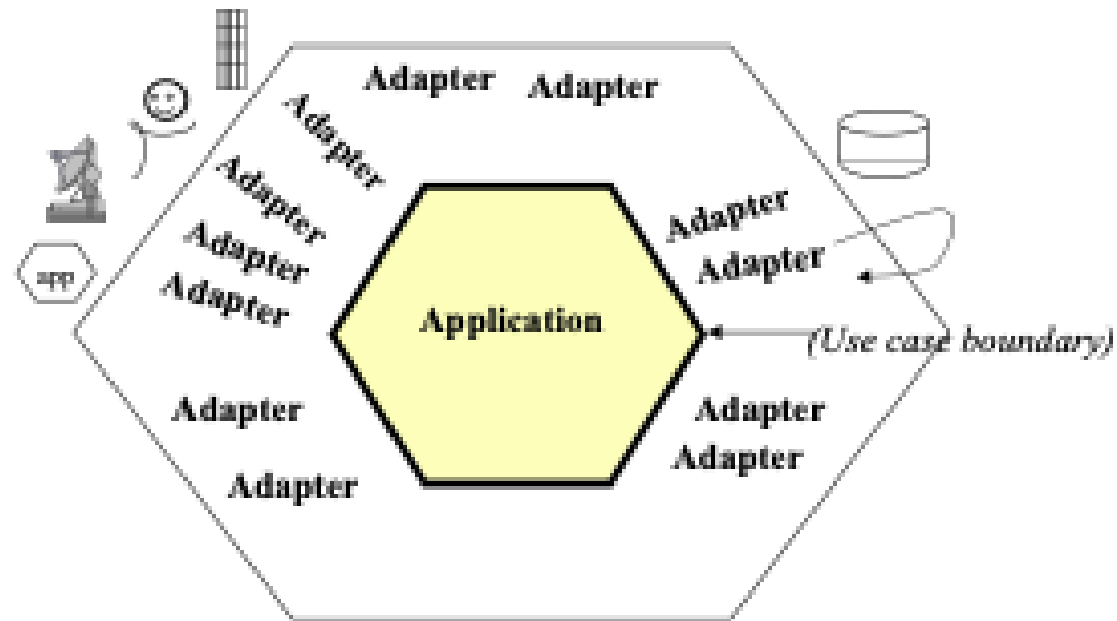
As we will also discuss later, the monolithic architecture is not a bad choice by itself.

It has some limitations (but no architecture is perfect; they all have strengths and weaknesses), but **it's a good fit for many apps.**

# MONOLITHIC ARCHITECTURE

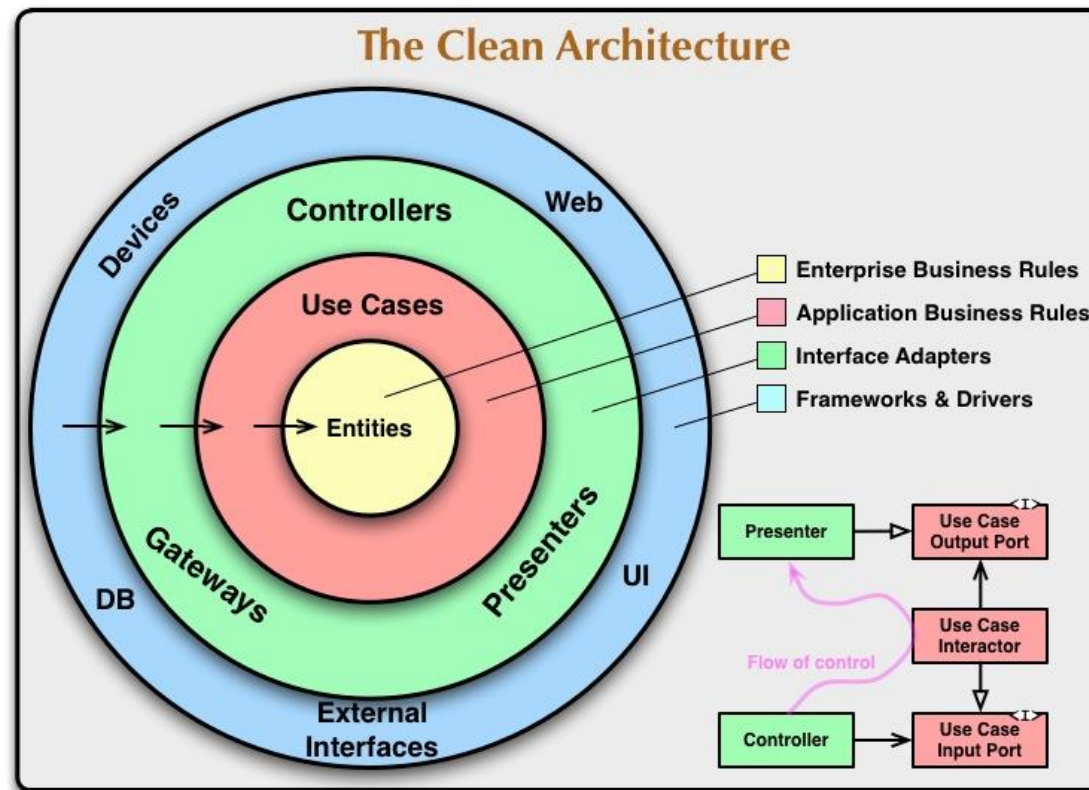
Still, it would be nice to evolve towards cleaner architecture, which ensures separation of concerns across the entire app...

# HEXAGONAL ARCHITECTURE



[The Hexagonal \(Ports & Adapters\) Architecture](#) – Alistair Cockburn

# THE CLEAN ARCHITECTURE



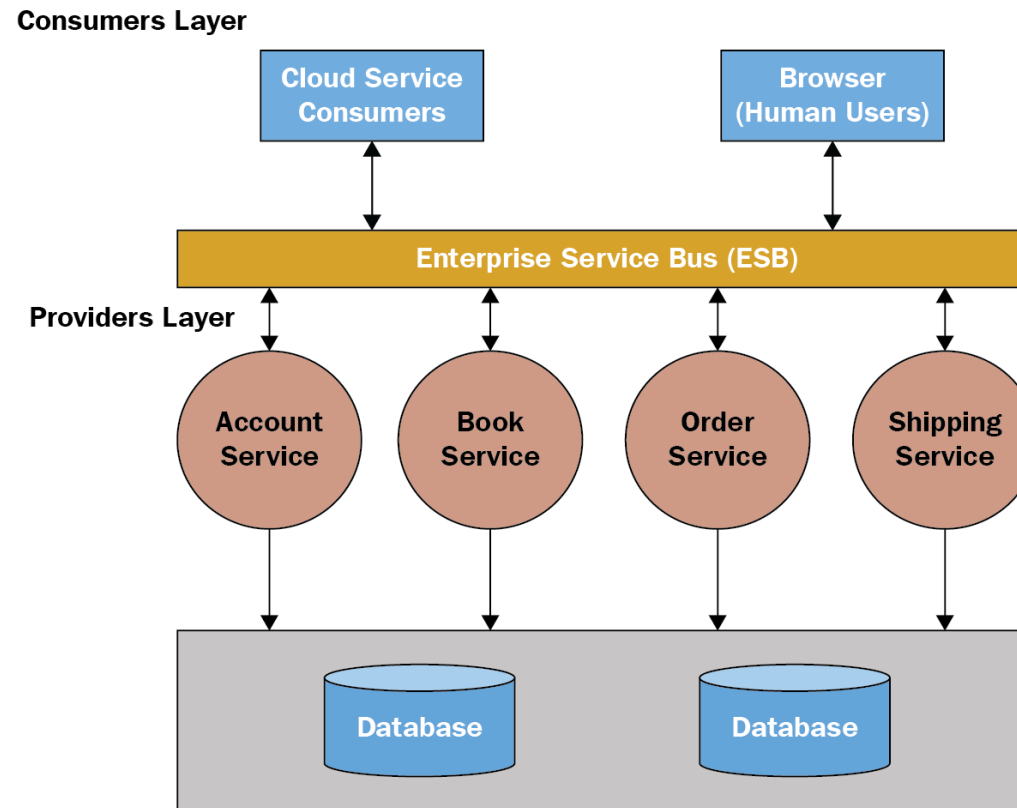
The Clean Architecture – Robert C. Martin (Uncle Bob)

# SERVICE-ORIENTED ARCHITECTURE

A **Service-Oriented Architecture** (SOA) is an architectural style in which the application is composed of multiple independent services, instead of a single monolithic app.

These services could be **independent processes** running on the same computer. Usually, they are **web apps** (communicating through web APIs) hosted on the same or different servers.

# SERVICE-ORIENTED ARCHITECTURE

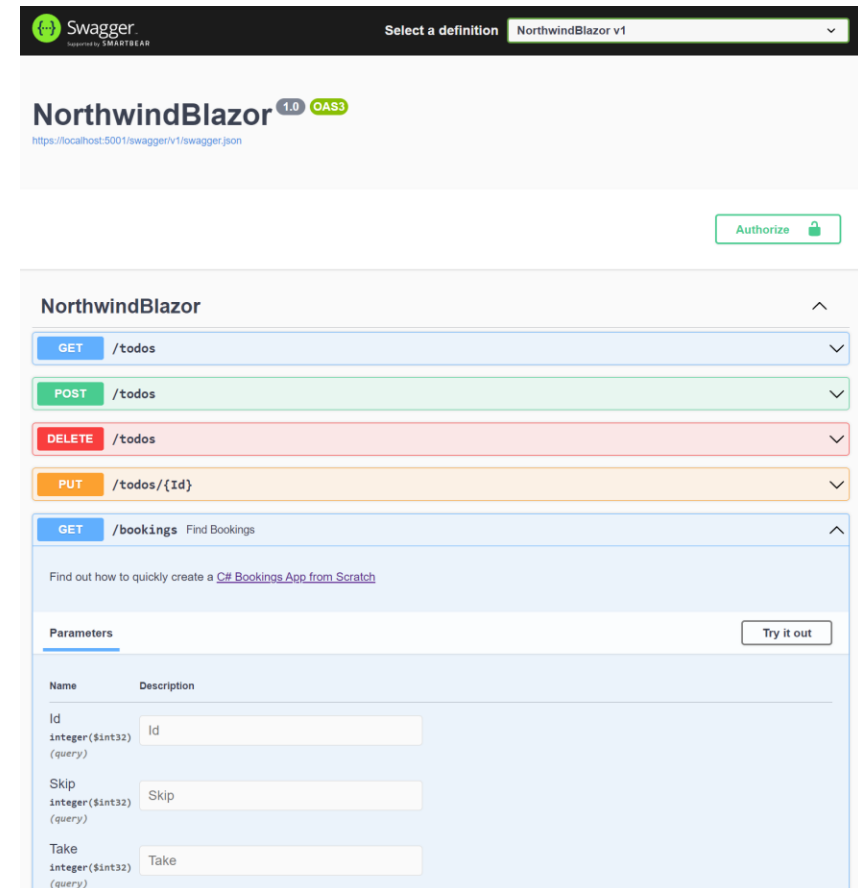


[Image source](#)



# PRINCIPLES OF SERVICE-ORIENTED ARCHITECTURE

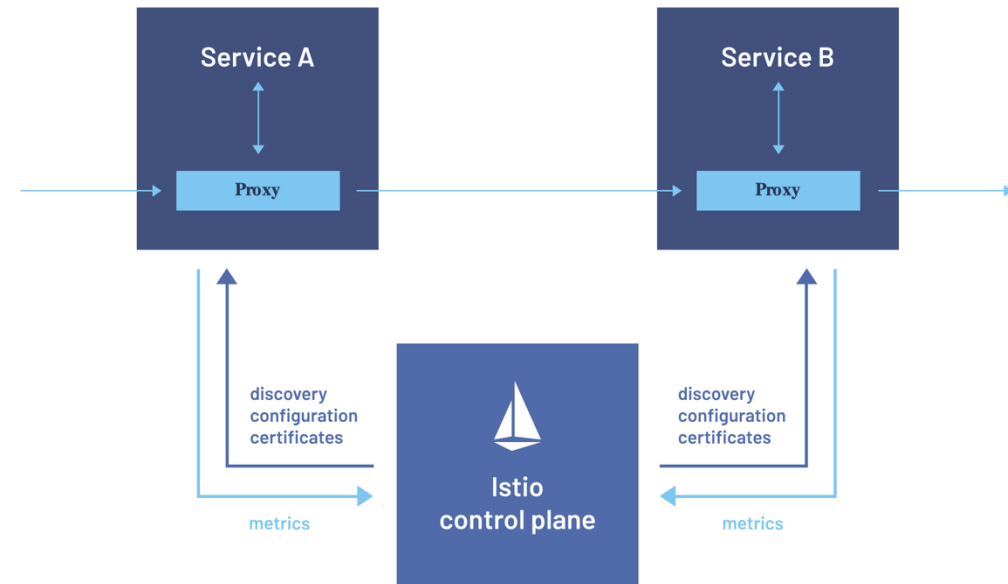
Each **service** is supposed to expose **a public (web) API** and associated **metadata**, describing its **contract**: API endpoints, parameter passing conventions, authentication and authorization, the set of preconditions and postconditions which need to hold for calling a route, etc.



# SERVICE MESH

When you have a lot of services, you need an appropriate infrastructure to connect and manage them.

A **service mesh** acts as a **proxy** between services, providing features such as **security** (encryption), **observability** (monitoring), **automatic retries** and exponential backoff for failed requests etc.



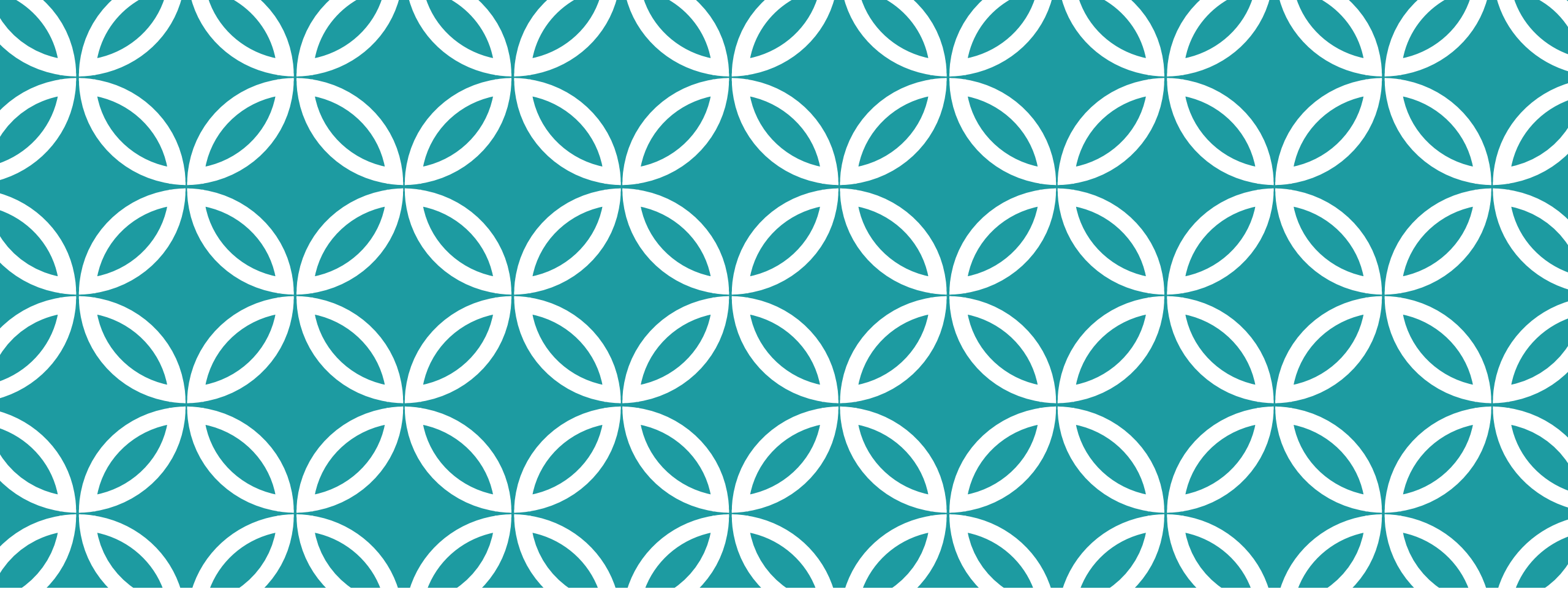
Istio, a service mesh implementation

# SERVICE-ORIENTED ARCHITECTURE IN PRACTICE

In theory, a service-oriented architecture **solves most or all the problems** we've discussed earlier with our simple monolithic client-server architecture.

In practice, the way SOA was implemented by various vendors fell short of these goals.

Microservices, which we will introduce next, are heavily inspired by SOA, but they **do it better**.



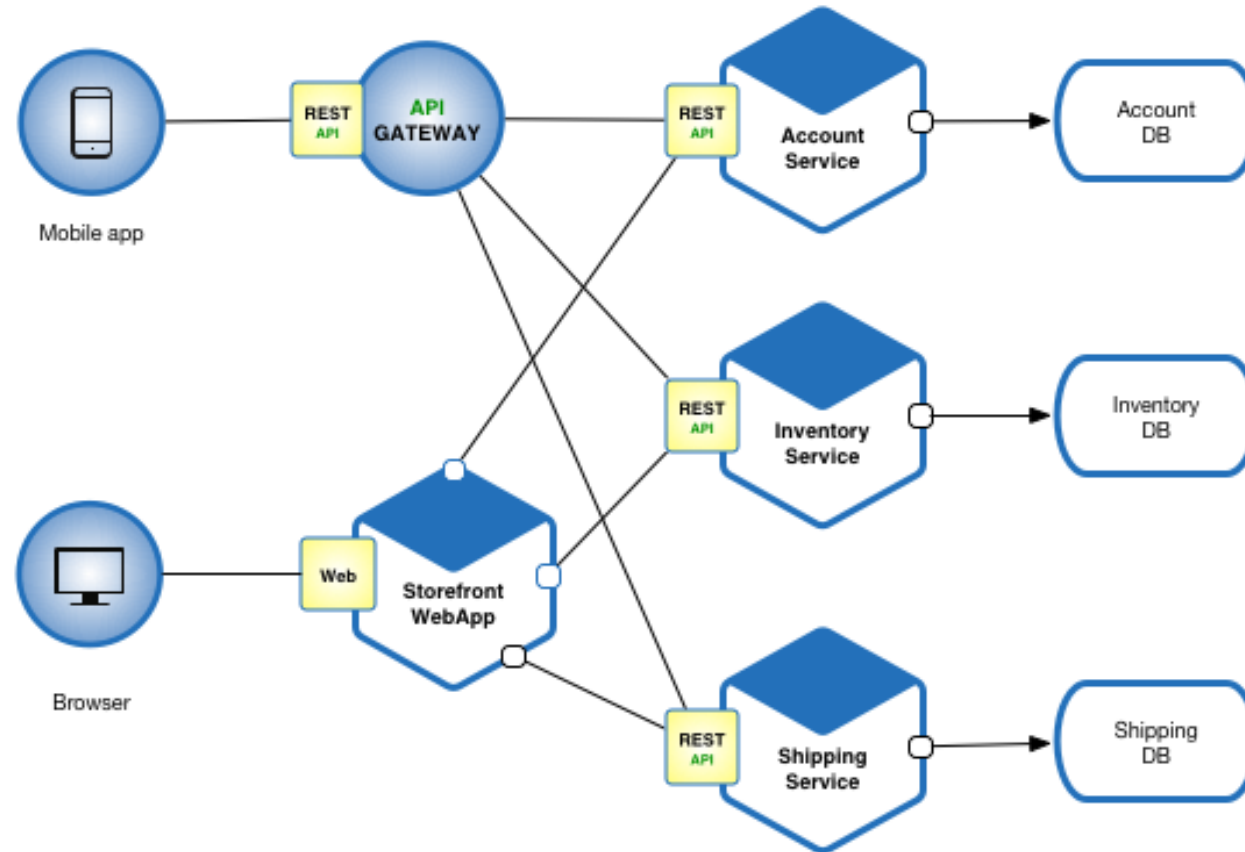
**MICROSERVICES** |

# MICROSERVICES

**Microservices** are encapsulated packages of business functionality, which handle one business/domain feature end-to-end.

They are meant to be **independently deployable**, **own their state**, be fairly **small** and **specialized**, **hide implementation details**, possibly handle their own **authentication/authorization** etc.

# MICROSERVICES ARCHITECTURE



[Image source](#)

# EXAMPLES OF MICROSERVICES

## **Examples:**

- Products list service
- Shopping cart service
- Mail sending service

## **Counterexamples:**

- Database service (layer)
- Authorization service (layer)

**Remember:** microservices are modeled along business models, not based on technical considerations.

# SINGLE RESPONSIBILITY

Every microservice should be **responsible** for a **single business entity** (and eventually the associated sub-entities, if they cannot exist by themselves).

**Good:** Books service, Clients service, Reviews service

**Bad:** "Data" service (handling all entities), "Cache" service (caching all requests), Review reply service (why not handle them in the Reviews service?)



# INDEPENDENTLY DEPLOYABLE AND MAINTAINABLE

Since the services are decoupled and interact only through API calls, you should be able to **update** one of the services **without** bringing down the whole app (although some features might be unavailable until the upgrade is complete, unless you perform a rolling release).

This ensures **faster development iteration times** with **less annoyance** for the **customers**.

# INDEPENDENTLY DEPLOYABLE AND MAINTAINABLE

**3.14.0**  
MAJOR *Minor* path

One way to avoid issues with frequently redeploying/updating services is to use **semantic versioning for the API routes**.

Example: the API routes are initially behind the `/v1/` prefix (e.g. `/api/v1/new`, `/api/v1/list`, `/api/v1/delete` etc).

The **schema gets updated** and **new fields** are added to the model.

The new model is accessible behind the `/v2/` prefix (e.g. `/api/v2/create`, `/api/v2/query`, `/api/v2/delete` etc). The v1 interface still works, but the route handlers translate the data to the new model behind the scenes.

# MICROSERVICES OWN THEIR DATA

Each microservice should either be **stateless** (in which case, every instance of it should react the same to the same inputs) or **stateful** (in which case, it should have its own private database / data store).

Yes, this might mean multiple independent instances of the same data storage software! (multiple PostgreSQL instances, multiple Redis instances, etc)

# MICROSERVICES OWN THEIR DATA

For example, imagine a **Books service**, which owns the (relational) database of available book titles (e.g. PostgreSQL).

You could also have a **Shopping cart service**, which stores the current shopping cart (tied to the user's session) in a Redis database.

The **Mail notifications service** stores its background jobs in its own Redis instance, whereas the **Search service** uses Elasticsearch to index the data of the books it gets from the Books service (through API calls).

# MINIMAL INTERFACES

Microservices should **communicate** using **standard network protocols** (e.g. HTTP(S), gRPC, GraphQL etc) and try to **minimize** their **public APIs**.

The larger the public API is, the bigger the risk for other services to start depending on fields and functions which would better be left as implementation details of the other service.

# MINIMAL INTERFACES

**Example:** Image service with an API for: creating/uploading images, retrieving/downloading images and deleting images.

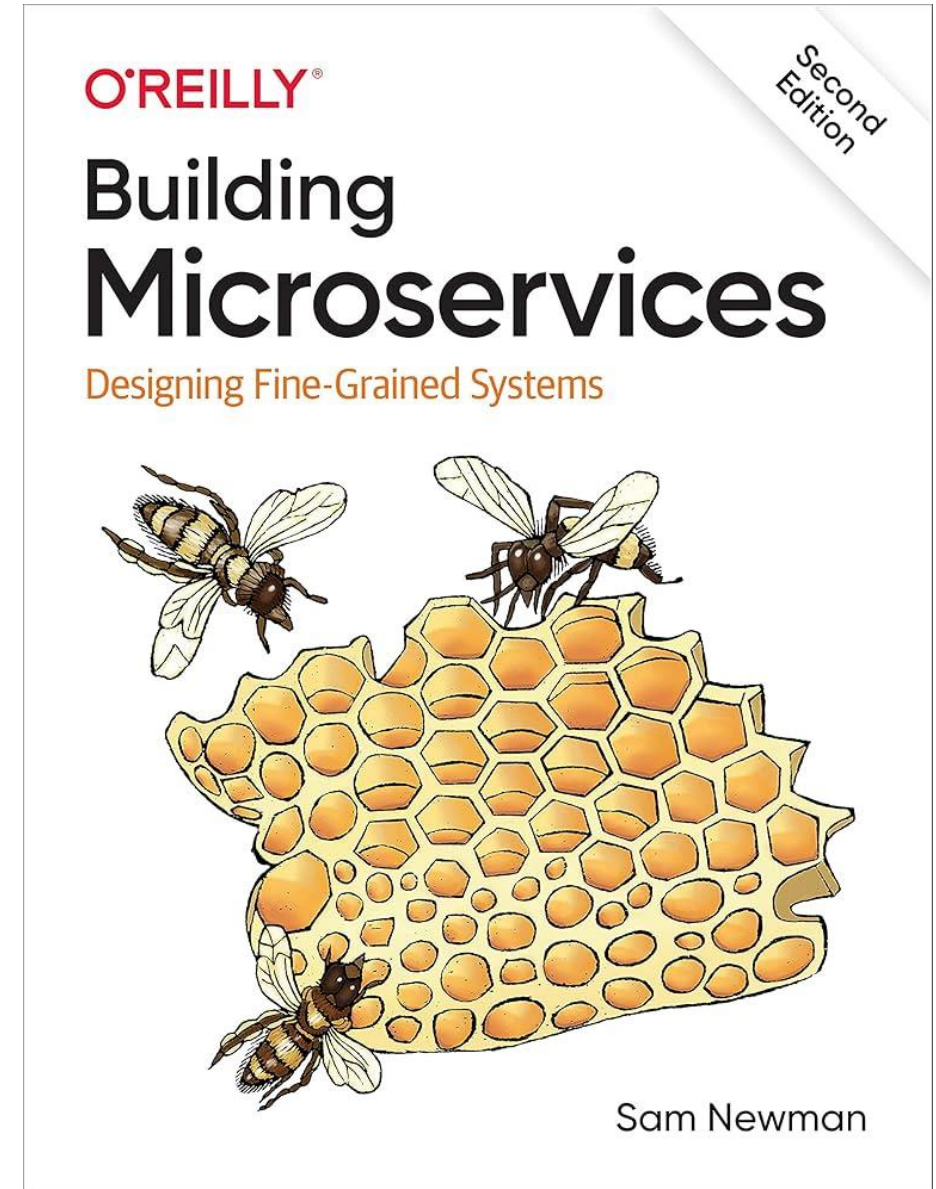
**Bad:** adding a route to the Image service for getting/setting which lossless compression algorithm it uses to store images (why would other services need to know this? What if you want to update it to use a better algorithm in the future?)

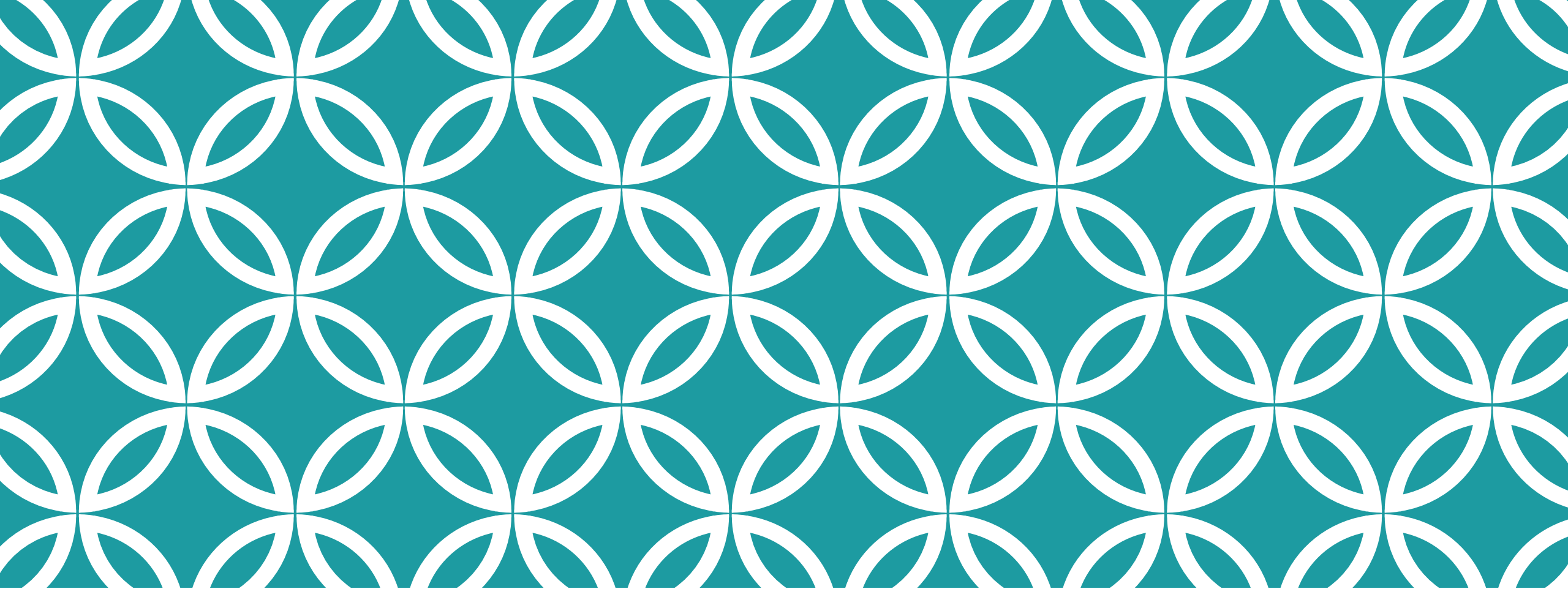
# FURTHER REFERENCES

Martin Fowler's [Microservices Guide](#)

[Building Microservices](#) book, by Sam Newman

[Microservices.io](#) site





# KUBERNETES

The magic making microservices  
happen!



# INTRO TO KUBERNETES

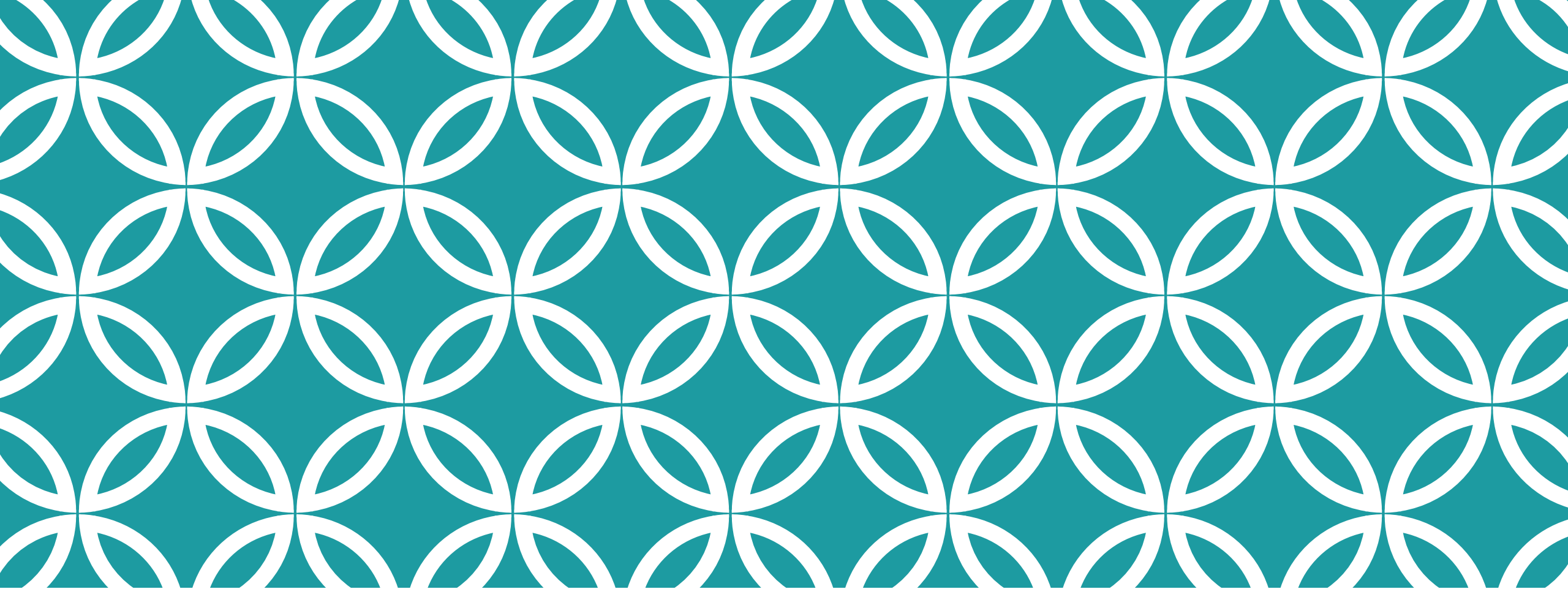
Kubernetes (also known for short as K8) is an open-source container orchestration system for automating their deployment, scaling and management.

Switch to K8s slide deck...

# KUBERNETES ON A STICK

How can you run Kubernetes on your own laptop?

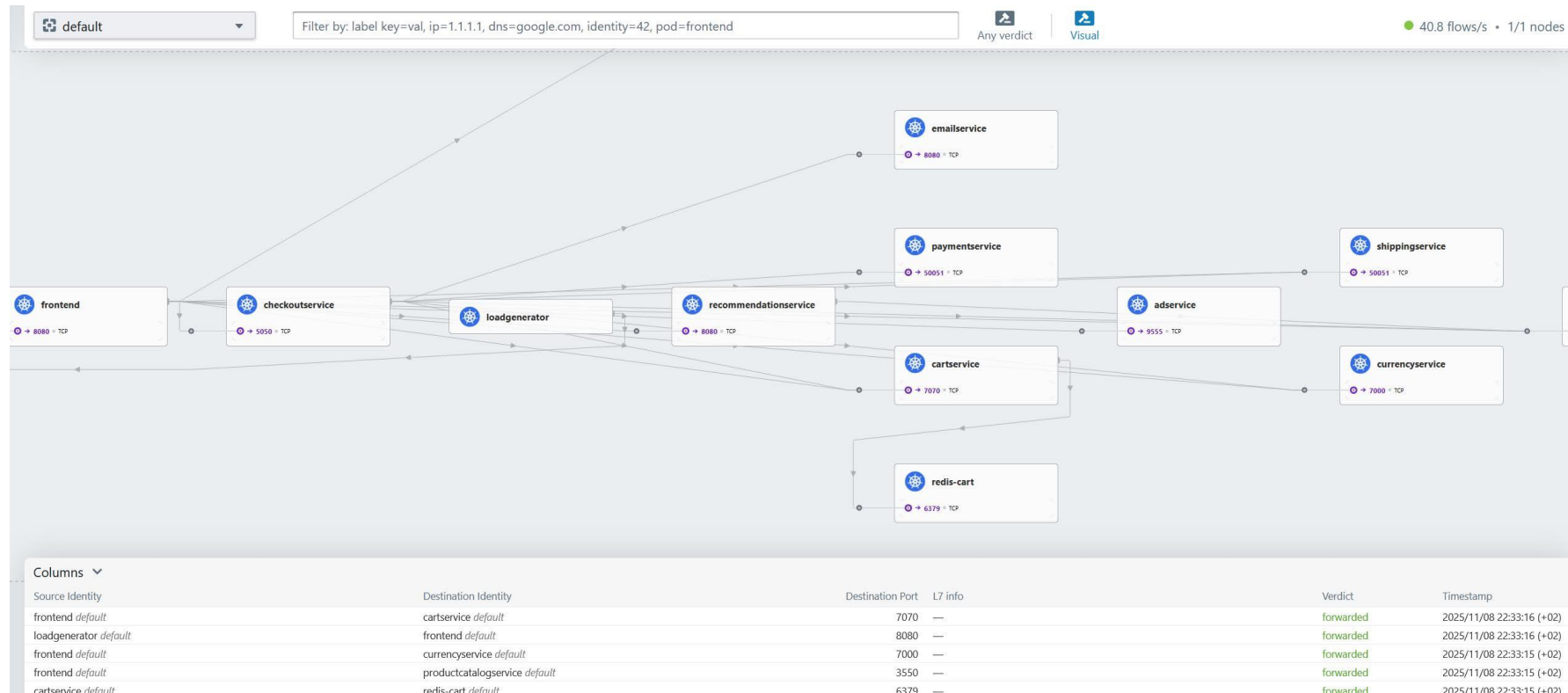
Simplest solution: use a distribution such as [Minikube](#)



# MICROSERVICES DEMO

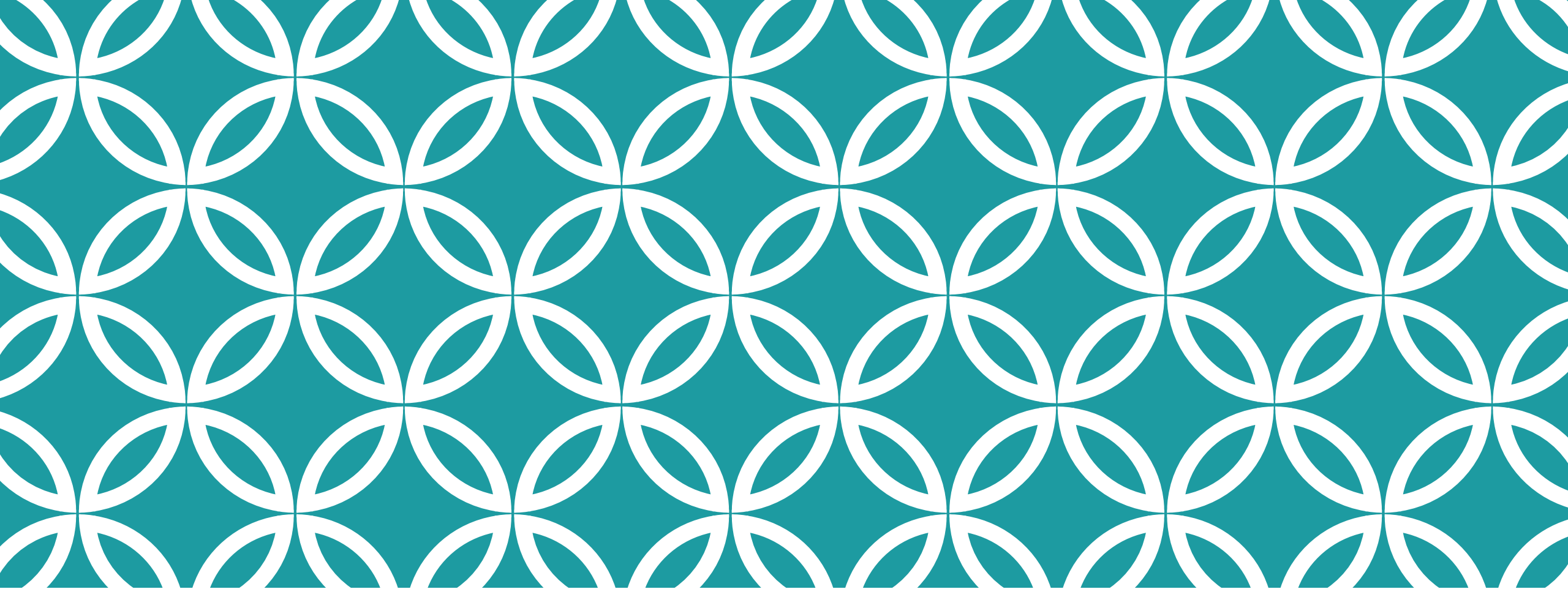
Time to roll our sleeves and get hacking!

# MICROSERVICES DEMO



Code: [github.com/GoogleCloudPlatform/microservices-demo](https://github.com/GoogleCloudPlatform/microservices-demo)

Need to install [Cilium](#) and [Hubble](#) to obtain this view



# DISADVANTAGES OF MICROSERVICES

When is it a bad idea to use them?

# DISADVANTAGES OF MICROSERVICES

Microservices work well for **large teams** or in specific use cases

They are probably the only way to ensure **high developer velocity** and **productivity** in **very large corporations**

They introduce a lot of **additional cost** and **complexity** for small apps and teams that don't need them!

# MICROSERVICES VS. THE MONOLITH

## The Majestic Monolith



[DHH](#) / FEBRUARY 29, 2016 / [3 COMMENTS](#)



David Heinemeier Hansson, creator of the popular Ruby on Rails web application framework on [why microservices should not be the go-to architecture for all apps](#)

# MICROSERVICES VS. THE MONOLITH

This is a great pattern. No, really. Not being sarcastic here. **If you're Amazon or Google** or any other software organization with **thousands of developers**, it's a wonderful way to parallelize opportunities for improvement. Each service can be its own team with its own timeline, staff, and objectives. It can evolve independently, at least somewhat, of whatever else the rest of the constellation is doing.

As I said, all that pain is worth it **when you have no choice**. But most people do have a choice, and they do have an alternative. So allow me to present just one such choice: **The Majestic Monolith!**



But what if you're Amazon,  
and microservices are nevertheless **not**  
the right solution?

# ANECDOTE: HOW AMAZON PRIME VIDEO WENT FROM MICROSERVICES BACK TO A MONOLITH

Video Streaming

## Scaling up the Prime Video audio/video monitoring service and reducing costs by 90%

The move from a distributed microservices architecture to a monolith application helped achieve higher scale, resilience, and reduce costs.

Marcin Kolny

Mar 22, 2023

At Prime Video, we offer thousands of live streams to our customers. To ensure that customers seamlessly receive content, Prime Video set up a tool to monitor every stream viewed by customers. This tool allows us to automatically identify perceptual quality issues (for example, block corruption or audio/video sync problems) and trigger a process to fix them.

---

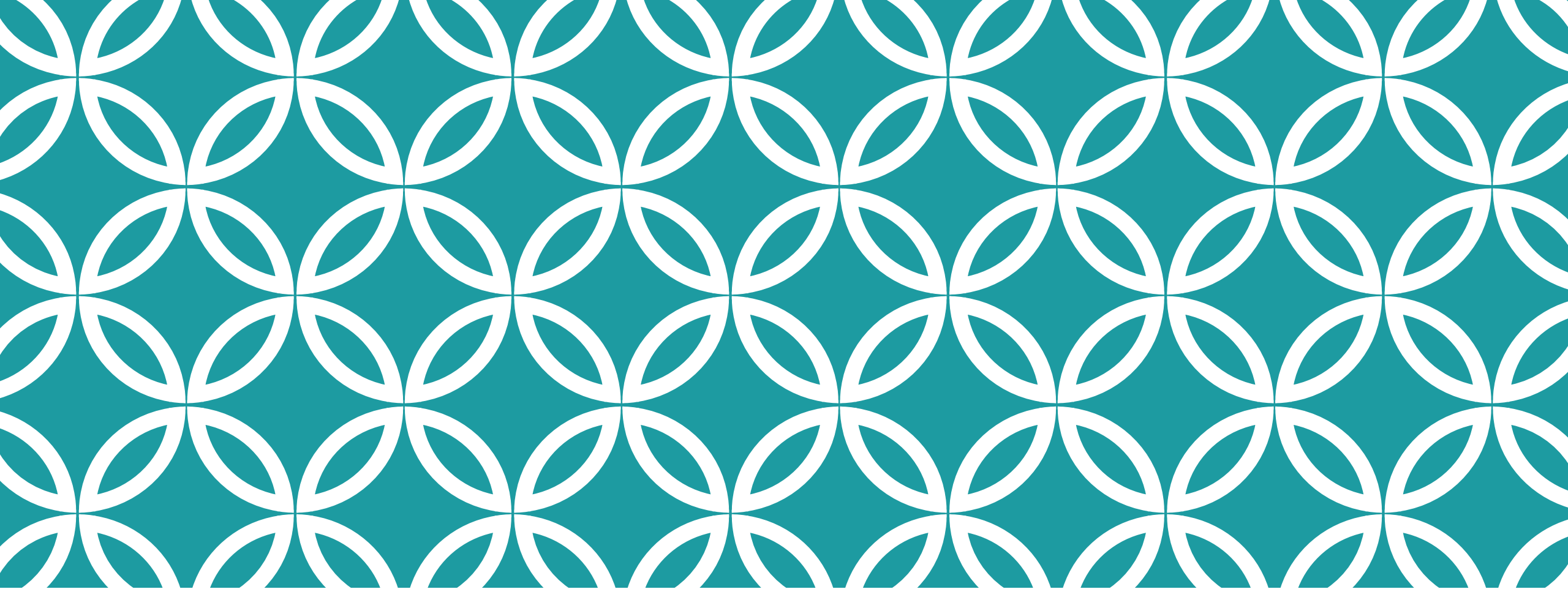
Most popular

**"We're just beginning to build the future of live sports streaming"**

Feb 07, 2023

---

[Amazon Prime Video technical blog post on why they went back to a monolith app](#)



# QUIZ TIME!

Kahoot!

The image features the Kahoot! logo in a bold, white, sans-serif font. The text is centered against a dark purple background. A lighter purple, semi-transparent geometric shape, resembling a stylized 'K' or a large arrow, is positioned behind the text, pointing towards the right. The exclamation mark at the end of the word is particularly prominent.

**Kahoot!**

Questions?

Comments?

# Thank you for your attention!



The illustrated children's guide to Kubernetes